# ATEC2026 Online Competition Contestant Guide

**Version Date: May 1, 2026**

## Guide Notes

- This Guide serves as the unified technical operations manual for both Track 1 (Locomotion-Priority) and Track 2 (Manipulation-Priority) of the 6th ATEC (ATEC2026) Online Simulation Challenge, covering task descriptions, environment configuration, interface specifications, submission procedures, and scoring rules.
- The two tracks share the same simulation environment and submission platform; the only difference lies in the L0-stage entry tasks. Track 1 uses Task A (Robot Hiking) as its L0 entry task, focusing on Legged Locomotion; Track 2 uses Task E (Table Clean-Up) as its L0 entry task, focusing on Manipulation. The L1-stage Whole-Body Manipulation tasks — Task B and Task D — are shared by both tracks, and the final ranking shall be determined by L1 results.
- For information on the competition schedule, prize structure, team-formation rules, intellectual property, and legal terms, please refer to the ATEC2026 official website and the *Participant Agreement*.

ATEC2026 CONFIDENTIAL

## Task Introduction

Welcome to the ATEC2026 Simulation Challenge. The ATEC2026 Simulation Challenge is divided into two progressive stages: L0 (Entry Stage) and L1 (Advanced Stage). A participant shall be irrevocably deemed to have successfully completed an L0 task only after submitting at least once and obtaining a score of **0.01 or above**, which is a prerequisite for L1 qualification. All tasks must be completed by the robot in a fully autonomous manner.

### L0 Stage — Entry Tasks (Single-Capability)

The L0 stage focuses on fundamental robotic capabilities, including locomotion and manipulation.

#### Task A: Robot Hiking

In this task, the robot must traverse a course of approximately 400 meters that includes diverse terrain conditions. Terrain types include:

- Flat ground
- Uneven terrain
- Inclined surfaces (uphill and downhill)
- Multi-step staircases (ascent and descent)

This task primarily evaluates the robot's locomotion stability and terrain adaptability under varied environmental conditions.

### Task E: Table Clean-Up

This task evaluates the robot's fundamental manipulation capability in a structured tabletop environment. The robot must:

- Grasp objects from the table surface
- Handle 3 different categories of objects
- Place the objects into a designated pink target container

This task primarily evaluates the robot's perception robustness and grasping accuracy.

## L1 Stage — Advanced Tasks (System Integration)

The L1 stage introduces more complex problems requiring tight integration of robotic locomotion and manipulation capabilities.

### Task B: Trash Collection and Disposal

In this task, the robot operates within a 20×20 meter workspace and must:

- Collect 18 target objects
- Handle multiple object categories
- Place the objects into the designated area

This task evaluates the robot's performance in long-horizon mobile manipulation and autonomous planning within a cluttered environment.

### Task D: Obstacle Crossing

This task focuses on advanced locomotion capability and environmental interaction. The robot must:

- Traverse challenging structures, including elevated platforms and ditches
- Modify the environment when necessary (e.g., using objects to fill ditches)
- Successfully reach the designated goal location

This task evaluates the robot's spatial understanding and long-horizon reasoning capability.

## Robot Platform

This Challenge provides multiple standardized robot platforms to support the locomotion and manipulation tasks involved in the competition. A total of five distinct robots are provided, covering a broad range of locomotion and manipulation capabilities.

ATEC2026 COMMITTEE

**Robot platforms:**

- **Manipulator**: A lightweight fixed-base 6-DoF manipulator equipped with a 2-DoF gripper, used for Task E.
- **Quadruped with Manipulator**: A quadruped mobile platform with a lightweight 6-DoF manipulator, used for Task A, Task B, and Task D.
- **Wheel-Legged Quadruped with Manipulator**: A four-wheel-legged mobile platform with a 6-DoF manipulator, used for Task A, Task B, and Task D.
- **Wheeled-Biped with Manipulator**: A two-wheel-legged mobile platform with a 6-DoF manipulator, used for Task A, Task B, and Task D.
- **Humanoid Robot**: A 29-DoF humanoid robot equipped with a 2-DoF gripper, used for Task A, Task B, and Task D.

**Sensor configuration:**

All robots are equipped with a standardized sensor suite to ensure cross-task and cross-platform consistency:

- **RGB Camera**: For visual perception and scene understanding
- **Depth Camera**: For 3D geometric reconstruction and distance estimation
- **LiDAR**: For terrain perception and environmental modeling
- **Contact Sensor**: For collision detection

ATEC2026 COMMITTEE

## Evaluation Metrics

### Task A Scoring

Evaluation is based on terrain traversal and task completion:

- Passing flat terrain: **+2 points**
- Passing rugged terrain: **+4 points**
- Passing slopes: **+8 points**
- Passing stairs: **+8 points**
- Reaching the finish line: **+4 points**

**Maximum total score: 26 points.**

### Task E Scoring

Task E evaluates precise tabletop manipulation, including grasping and placement.

- Each successful grasp: **+3 points**
- Correct placement into the target area: **+3 points**

**Maximum total score: 18 points.**

## Task B Scoring

Task B evaluates the robot's ability to perform object collection and placement in a large-scale environment. For each object:

- Successful object pickup: **+1 point**
- Correct placement into the target area: **+1 point**

There are 18 objects in total, each contributing to the total score. **Maximum total score: 36 points.**

## Task D Scoring

Task D evaluates the robot's ability to traverse obstacles and interact with the environment to complete the task:

- Reaching the obstacle area's starting point: **+2 points**
- Successfully traversing an obstacle (platform or ditch): **+20 points**
- Using boxes to overcome elevated platforms or to bridge ditches (environmental modification): **+14 points**

**Maximum total score: 36 points.**

**Notes:**

- Scores across tasks are not directly comparable; the final ranking shall be determined by the L1 composite result.
- The scoring design is intended to balance task completion, complexity, and system stability.

This Guide provides the complete technical specification for the ATEC2026 Online Simulation Challenge, including task definitions, environment configuration, interface specifications, scoring rules, and submission procedures.

ATEC2026 COMMITTEE

**Quick Start:**

1. Install the Isaac Lab environment
2. Run the example tasks (`scripts/view_task_*.py`)
3. Modify `demo/solution.py`
4. Local validation
5. Submit for evaluation

**Typical Pipeline**

The competition workflow follows a standard closed loop from algorithm development to online evaluation, as outlined below:

**1. Develop & Train**

Conduct policy training in a local or cluster environment. Contestants may build upon the official simulation environment and baseline routines (such as RL / ACT), combining custom methods (such as reinforcement learning, imitation learning, or optimization-based control) for system development, model training, and optimization.

**2. Validate**

Validate the trained policy in a local environment using the official scripts (such as `scripts/play_atec_task.py`) for visual testing and debugging, ensuring stable execution across different tasks and robot platforms within time and resource constraints.

**3. Package**

Package the final solution in the standard submission format. Core requirements include:

- Implementing `demo/solution.py` as the unified entry point
- Preparing the model file (e.g., `policy.pt`) and dependencies (`requirements.txt`)
- Ensuring that the program can run independently in **a network-isolated environment**
- Controlling initialization time (startup shall be completed within an absolute hard cap of **300 seconds**)

**4. Submit**

Submit the solution via the platform:

- Method 1: Upload source code; the platform automatically builds the image
- Method 2: Build and push a Docker image yourself

Once submitted, the system shall automatically trigger the evaluation pipeline.

**5. Evaluate**

The evaluation system will:

- Launch the simulation environment and the contestant container
- Interact with the contestant's algorithm through the HTTP interface (`/step`)
- Run the complete task within the prescribed time (an absolute hard cap of **30 minutes wall-clock time**)
- Automatically compute the final score based on task completion and update the leaderboard

**6. Feedback and Improve**

Based on the score and leaderboard feedback, contestants continuously refine their algorithms and models.

ATEC2026 COMMITTEE

## Leaderboard Logic

The ATEC2026 Online Competition adopts a stage-based leaderboard mechanism to ensure fairness and to highlight system-level capability evaluation. The leaderboard consists of two parts — **L0 (Entry Stage)** and **L1 (Advanced Stage)** — but the two play different roles:

### 1. L0 Stage (Entry & Qualification Filtering)

- The L0 stage is used to evaluate fundamental capabilities (locomotion or manipulation).
- Each task (Task A / Task E) has its own independent leaderboard.
- A contestant unlocks the L1 stage as long as they obtain a valid score in any L0 task.
- L0 scores do not count toward the final ranking and are used solely for advancement-qualification determination.

### 2. L1 Stage (Final Ranking Basis)

- The L1 stage evaluates system-level capability (locomotion + manipulation + environmental interaction).
- It includes Task B (Trash Collection and Disposal) and Task D (Obstacle Crossing and Environmental Interaction).
- The final leaderboard shall be ranked based on the L1 composite performance.

**L1 total score formula:**

```
Final Score = Score(Task B) + Score(Task D)
```

- Ranking is sorted by total score in descending order.
- In the event of a tie, the following shall be compared in order:
  - Task B score (priority)
  - Task D score
  - Completion time (shorter is better)
  - If the Task B score, Task D score, and completion time are all identical, the Team that first achieved that score shall rank higher (determined by the timestamp of the final submission recorded in the backend system).

### 3. Submission and Leaderboard Updates

- After each successful evaluation, the system shall automatically update the leaderboard.
- Each Team retains only its highest historical score (Best Score) on the leaderboard.
- The leaderboard is updated in real time, and contestants may check their current ranking at any time.

### 4. Advancement Mechanism

ATEC2026 COMMITTEE

- The L1 leaderboard shall serve as the primary basis for advancement to the Real-World Preliminary.
- For specific advancement quotas and rules, please refer to the descriptions on the official competition website.

**Core Principle:**

The leaderboard emphasizes system-level capability (L1), encouraging contestants to progress from single-capability mastery (L0) to a complete task closed loop.

## Competition Environment

This repository is developed and tested on Isaac Lab v2.3.2.

### Installation and Setup

**Step 1: Install Isaac Lab v2.3.2**

Please install Isaac Lab by referring to its official documentation: https://isaac-sim.github.io/IsaacLab/v2.3.2/source/setup/installation/isaaclab_pip_installation.html.

**Step 2: Download the Code**

The competition code consists of two parts: `ATEC2026_Simulation_Challenge` and `atec_robot_model`. Among them, `ATEC2026_Simulation_Challenge` contains the simulation-environment code and the Demo code, while `atec_robot_model` contains non-text assets used by the simulation environment, such as scene assets, robot models, and ACT-demo training-result files. The code may be obtained via either GitHub or the official website.

**Method 1: Download via GitHub**

```
# Clone the repository via git
git clone https://github.com/atecup/ATEC2026_Simulation_Challenge

# Download atec_robot_model into the ATEC2026_Simulation_Challenge directory
cd ATEC2026_Simulation_Challenge
curl https://static.atecup.com/atec2026/atec_robot_model.zip -o atec_robot_model.zip
unzip atec_robot_model.zip -d atec_robot_model
```

**Method 2: Download via the Official Website**

On the submission page, click the “Resource Center” tab, then click the “Download Demo” button to begin downloading. The downloaded file is named `ATEC2026_Simulation_Challenge.zip`; simply extract it into the current directory.

**Note:** This directory already contains the contents of `atec_robot_model`; no additional download is required.

**Step 3: Install the ATEC Extension**

ATEC2026 COMMITTEE

```
cd ATEC2026_Simulation_Challenge/source/atec_rl_lab
pip install -e .
```

**Tip:** This command must be executed within the environment where Isaac Lab is installed (this depends on how Isaac Lab was installed).

After installation, all `ATEC-*` tasks shall be available within the corresponding Python virtual environment.

## Competition Environment List

The `atec_rl_lab.tasks` module registers all arena–robot combinations as Gym-compatible environments, providing a unified interface for evaluation and submission.

| Arena / Robot | G1 | Tron1+Piper | B2+Piper | B2w+Piper | Piper |
|---|---|---|---|---|---|
| Task A | ATEC-TaskA-G1 | ATEC-TaskA-Tron1Piper | ATEC-TaskA-B2Piper | ATEC-TaskA-B2wPiper | |
| Task B | ATEC-TaskB-G1 | ATEC-TaskB-Tron1Piper | ATEC-TaskB-B2Piper | ATEC-TaskB-B2wPiper | |
| Task D | ATEC-TaskD-G1 | ATEC-TaskD-Tron1Piper | ATEC-TaskD-B2Piper | ATEC-TaskD-B2wPiper | |
| Task E | | | | | ATEC-TaskE-Piper |

**Note:** The provided environments are designed exclusively for evaluation and submission and do not support parallel training.

ATEC2026 COMMITTEE

## Environment Check

```
cd ATEC2026_Simulation_Challenge
python scripts/list_envs.py

python scripts/view_robots.py --enable_cameras  # Inspect robot models
python scripts/view_task_a.py --enable_cameras  # Visualize Task A
python scripts/view_task_e.py --enable_cameras  # Visualize Task E
python scripts/view_task_b.py --enable_cameras  # Visualize Task B
python scripts/view_task_d.py --enable_cameras  # Visualize Task D
```

## Demo Description

### Directory Structure

```
ATEC2026_Simulation_Challenge/demo/
├── Dockerfile          # Example Dockerfile
```

```
├── __init__.py         # Used as a Python package only during local de
bugging
├── requirements.txt    # (Optional) Contestants may add dependencies i
n this file
├── run.sh              # (Required) Launch script used during platform
 scoring; shall not be modified
├── server.py           # (Required) HTTP service file; shall not be mo
dified
├── solution.py         # (Required) Core code of the contestant's solu
tion
├── solution_zero.py    # All-zero demo solution
├── solution_rl.py      # RL demo solution
├── policy.pt           # RL demo training-result file
├── solution_act.py     # ACT demo solution
├── act                 # Other code for the ACT demo
└── policy_act.pt       # ACT demo training-result file (contestants mu
st
                        # copy ../atec_robot_model/baseline/act/policy.
pt
                        # to policy_act.pt themselves)
```

## Implementation Requirements

### Overview

1. Modify `solution.py`
2. Implement `AlgSolution.predict()`
3. Return `action`

### Recommended workflow:

1. Use the baseline
2. Local validation
3. Submit

Contestants must implement `demo/solution.py`, and the file name shall not be changed. This file serves simultaneously as the entry point for `scripts/play_atec_task.py` (used for local validation) and `demo/server.py` (used for online submission). The three files `demo/solution_zero.py`, `demo/solution_rl.py`, and `demo/solution_act.py` are different solution files; in actual use, the chosen file must be renamed to `demo/solution.py`.

- **Class name:** `AlgSolution`
- **Function:** `predict(obs, current_score)`, where `obs` is the current robot state and `current_score` is the score currently obtained
- **Return value:** `{"action": action, "giveup": False}`, where `action` is the current robot control command (which must be provided as a `List`), and `giveup` is the give-up flag (if set to `True`, the task shall be terminated immediately)

ATEC2026 COMMITTEE

## Local Validation

```
python scripts/play_atec_task.py \
    --task ATEC-TaskA-G1 \
    --enable_cameras \
    --debug
```

When run, `scripts/play_atec_task.py` loads `demo/solution.py` as the sole entry point. Should a contestant wish to switch the solution, the solution file must be renamed to `demo/solution.py`, or the import logic in `scripts/play_atec_task.py` must be changed accordingly. The model files read by `demo/solution_rl.py` and `demo/solution_act.py` can be found in the `atec_robot_model` directory; contestants may change the file paths as they see fit.

**Argument descriptions:**

- `--task`: Task and robot selection
- `--debug`: Print runtime and score

## Environment Interaction Interface

### Robot State Feedback

Each task provides corresponding robot state feedback as a `dict`, which can be indexed via the keys listed below.

**(1) Robot Proprioceptive Feedback (Proprio)**

The robot's proprioceptive feedback provides information about the robot's state. Defined as follows:

- `base_lin_vel`: Absolute linear velocity of the base in the body frame
- `base_ang_vel`: Angular velocity of the base in the body frame
- `velocity_commands`: Desired base velocity, SE(2) (e.g., target linear and angular velocity)
- `projected_gravity`: Gravity vector projected into the body frame
- `joint_pos`: Relative joint positions (joint order preserved)
- `joint_vel`: Relative joint velocities (joint order preserved)
- `actions`: Control commands at the previous time step

**Access method:** `obs["proprio"]`

The order of `joint_pos`, `joint_vel`, and `actions` is fixed and consistent across all environments. The order follows the robot-specific joint configuration:

**Per-robot joint order:**

**B2Piper (Quadruped with Manipulator):**

ATEC2026 COMMITTEE

```
FR_hip_joint, FR_thigh_joint, FR_calf_joint,
FL_hip_joint, FL_thigh_joint, FL_calf_joint,
RR_hip_joint, RR_thigh_joint, RR_calf_joint,
RL_hip_joint, RL_thigh_joint, RL_calf_joint,
arm_joint1, arm_joint2, arm_joint3, arm_joint4,
arm_joint5, arm_joint6, arm_joint7, arm_joint8
```

**B2wPiper (Wheel-Legged Quadruped with Manipulator):**

```
FR_hip_joint, FR_thigh_joint, FR_calf_joint,
FL_hip_joint, FL_thigh_joint, FL_calf_joint,
RR_hip_joint, RR_thigh_joint, RR_calf_joint,
RL_hip_joint, RL_thigh_joint, RL_calf_joint,
FR_foot_joint, FL_foot_joint, RR_foot_joint, RL_foot_joint,
arm_joint1, arm_joint2, arm_joint3, arm_joint4,
arm_joint5, arm_joint6, arm_joint7, arm_joint8
```

**Tron1Piper (Wheeled-Biped with Manipulator):**

```
abad_L_Joint, hip_L_Joint, knee_L_Joint,
abad_R_Joint, hip_R_Joint, knee_R_Joint,
wheel_L_Joint, wheel_R_Joint,
arm_joint1, arm_joint2, arm_joint3, arm_joint4,
arm_joint5, arm_joint6, arm_joint7, arm_joint8
```

**G1 (Humanoid Robot):**

```
left_hip_pitch_joint, left_hip_roll_joint, left_hip_yaw_joint,
left_knee_joint, left_ankle_pitch_joint, left_ankle_roll_joint,
right_hip_pitch_joint, right_hip_roll_joint, right_hip_yaw_joint,
right_knee_joint, right_ankle_pitch_joint, right_ankle_roll_joint,
waist_yaw_joint, waist_roll_joint, waist_pitch_joint,
left_shoulder_pitch_joint, left_shoulder_roll_joint, left_shoulder_yaw_
joint,
left_elbow_joint, left_wrist_roll_joint, left_wrist_pitch_joint, left_w
rist_yaw_joint,
right_shoulder_pitch_joint, right_shoulder_roll_joint, right_shoulder_y
aw_joint,
right_elbow_joint, right_wrist_roll_joint, right_wrist_pitch_joint, rig
ht_wrist_yaw_joint,
left_hand_Joint1_1, left_hand_Joint2_1,
right_hand_Joint1_1, right_hand_Joint2_1
```

**Piper (Fixed-Base Manipulator):**

```
joint1, joint2, joint3, joint4,
joint5, joint6, joint7, joint8
```

### (2) Image Feedback (Image)

The image feedback group provides multi-view visual input from on-board cameras to support perception in navigation and manipulation tasks:

ATEC2026 COMMITTEE

- `head_rgb`: RGB image from the head-mounted camera (global scene view)
- `head_depth`: Depth image from the head-mounted camera
- `ee_rgb`: RGB image from the end-effector camera (manipulation view)
- `ee_depth`: Depth image from the end-effector camera
- `ee_dual_rgb`: RGB image from the dual end-effector cameras (available only on the humanoid platform)
- `ee_dual_depth`: Depth image from the dual end-effector cameras (available only on the humanoid platform)
- `video_rgb`: RGB image from the tabletop (eye-out-of-hand) camera (available only for Task E)
- `video_depth`: Depth image from the tabletop (eye-out-of-hand) camera (available only for Task E)

**Access method:** `obs["image"]`

**(3) Exteroceptive Feedback (Extero)**

The exteroceptive feedback group provides information about the external environment, primarily for terrain perception and navigation:

- `lidar_scan`: LiDAR-based terrain point-cloud information obtained from the on-board LiDAR sensor.

**Access method:** `obs["extero"]`

### Robot Control

All robots in this Challenge are controlled via joint-position commands. The robot control command is defined as follows:

```
joint_pos = mdp.JointPositionActionCfg(
    asset_name="robot",
    joint_names=[".*"],
    scale=0.5,
    use_default_offset=True,
    preserve_order=True,
)
```

The control command shall be scaled by a factor of 0.5 before being applied to the robot. The action space is identical across all tasks, and the joint order is fixed, so no joint reordering is required.

## Example Solutions

This section provides example solutions for locomotion and manipulation tasks, including training, evaluation, and deployment workflows.

We provide two examples:

AIRC'2026 COMMITTEE

- Reinforcement Learning (RL) for locomotion
- Imitation Learning (ACT) for manipulation

Both follow a unified workflow: **train → evaluate → local validation → online submission**.

### (1) Reinforcement-Learning Example

For locomotion and mobile-manipulation tasks (Task A, Task B, Task D), a PPO-based reinforcement-learning (RL) example is provided, implemented in `atec_rl_lab.train.locomotion`.

**Training:**

From the repository root, run:

```
python scripts/rsl_rl/train.py \
    --task ATEC-Isaac-Velocity-Flat-Unitree-B2-v0 \
    --headless \
    --video
```

- `--headless`: Disable visualization
- `--video`: Save video

**Evaluation:**

After training, evaluate the policy with:

```
python scripts/rsl_rl/play.py \
    --task ATEC-Isaac-Velocity-Flat-Unitree-B2-v0
```

The following command may be used for local validation. Note that `demo/solution_rl.py` must be renamed to `demo/solution.py` before use.

```
cd ATEC2026_Simulation_Challenge
python scripts/play_atec_task.py \
    --task ATEC-TaskA-B2Piper \
    --enable_cameras \
    --debug
```

ATEC2026 COMMITTEE

### (2) Imitation-Learning Example

For the manipulation task (Task E), an Imitation-Learning example based on ACT (Action Chunking with Transformers) is provided, implemented in `atec_rl_lab.train.act`.

**Collect demonstration data:**

```
python scripts/act/collect_demos_task_e.py \
    --pick_objects 3 --num_demos 100 \
    --headless --enable_cameras --save_images
```

**Filter the dataset:**

```
python scripts/act/filter_demos.py \
    --input datasets/atec_task_e/trajectory.hdf5 \
    --output datasets/atec_task_e/trajectory_filtered.hdf5 \
    --threshold 0.001
```

**Train the policy:**

```
cd scripts/act
bash baseline.sh
```

The following command may be used for local validation. Note that `demo/solution_act.py` must be renamed to `demo/solution.py` before use.

```
cd ATEC2026_Simulation_Challenge
python scripts/play_atec_task.py \
    --task ATEC-TaskE-Piper \
    --enable_cameras \
    --debug
```

ATEC2026 COMMITTEE

## Online Scoring

During online scoring, the contestant's solution and the simulation environment shall run in two separate containers, with the scoring program responsible for relaying information between them. The contestant's solution is wrapped as an HTTP server (refer to `server.py`), and the scoring program interacts with the solution container via HTTP and with the simulation environment via the Gym API.

After the contestant's solution container starts, the platform probes the `/health` endpoint to determine whether the HTTP service has started successfully, with an absolute hard cap of **300 seconds (UTC+8 reference clock)** — that is, contestants must complete code initialization and bring the server up within 300 seconds. Failure to return a successful `/health` response within 300 seconds shall be irrevocably deemed a service-startup failure.

The scoring program first initializes the simulation environment via Gym's `env.reset()`, obtains the initial simulation environment information and the robot's observation information, and calls the `/step` endpoint to pass this information to the contestant's solution. The solution must, based on this information, predict the robot's next action command and return it synchronously. The scoring program shall pass the action command to the simulation environment via Gym's `env.step()`, obtain the new simulation environment information and robot observation information, and again call `/step` to pass them to the solution. This loop continues iteratively.

The scoring program starts timing from `env.reset()`. After **30 minutes (an absolute hard cap of wall-clock time)**, execution shall be forcibly terminated, and the current score shall be returned as the contestant's final score.

After the contestant's solution terminates, the scoring program shall call the HTTP `/quit` endpoint to notify the contestant container to exit.

## Submission Methods

### Overview

Upload methods:

1. Source-code upload
2. Image upload

**Daily submission limits (reset at 10:00 UTC+8):**

- **Submission count:** an absolute hard cap of **10 attempts** per day (including failures)
- **Scoring opportunities:** an absolute hard cap of **3 results** per day (successful evaluations only)

**Rules:**

- Build failure → not counted toward submissions
- Startup failure → counted toward submissions
- Successful scoring → counted toward both

ATEC2026 COMMITTEE

### Method 1: Source-Code Upload

This Competition supports the source-code upload method. You only need to submit the source code, and the system shall automatically complete image building and scoring.

**Operating procedure:**

1. Enter the submission page and select the "Upload Source Code" tab.
2. Select the robot configuration.
3. Upload the files:
   - Click the "Upload File" or "Upload Folder" button to select local code and model files.
   - Click the "Submit Changes" button to begin uploading.
4. Confirm the directory structure. The root directory shall follow the structure below (refer to the Demo's directory structure):

```
├── solution.py          # (Required) Core code of the contestant's solu
tion
├── policy.pt            # (Optional) Training-result file
├── requirements.txt     # (Optional) Contestants may add dependencies i
```

```
n this file
└── Other files         # (Optional) Other files
```

5. Click the “Submit for Scoring” button. The system shall execute the image-building operation and trigger scoring upon successful build.

**Notes:**

- The base image environment is Python 3.12, PyTorch 2.7.1, CUDA 12.8.1, with Alinux 3 (2104) as the operating system.
- The Dockerfile used for the build is referenced from the `Dockerfile` under the `demo` directory.

**Important Notices:**

- Please do not upload the `run.sh` and `server.py` files; the system shall insert these two files during image building, and their content is identical to those in the Demo.
- Please refer to the *Appendix: List of Pre-installed Dependencies in the Environment* to avoid dependency conflicts.
- The number of files is strictly and absolutely prohibited from exceeding 300.

### Method 2: Image Push

This Competition supports the image-push submission method. You build the Docker image locally, push it to the designated repository, and the system shall automatically begin scoring.

**Operating procedure:**

1. Enter the submission page and select the “Push Image” tab.
2. Select the network access point and the robot configuration.
3. Click the “Generate Temporary Credentials” button. Three Docker commands (login, build, push) shall be generated.
4. Execute the three commands in order in your code directory to push the image to the ATEC server.

**Important Notices:**

- Your directory structure must remain consistent with the Demo’s directory structure; if it does not, the Docker commands must be adjusted accordingly.
- The image size is subject to an absolute hard cap of **30 GB**. If exceeded, the submission record’s status shall be “Pre-check Failed”; clicking the exclamation icon on the right reveals the cause. This status does not deduct from the submission count.

**Description of the three Docker commands:**

ATEC2026 COMMITTEE

```
# Command 1: Log in to the Docker image repository
docker login --username=cr_temp_user ******

# Command 2: Build the Docker image
docker build -t ******

# Command 3: Push to the image repository
docker push ******
```

## Submission Limits

- This Competition imposes a dual-quota control mechanism on submission frequency, with a 24-hour timing cycle (10:00 on day T+0 to 10:00 on day T+1, **UTC+8**).
- **Quota 1: "Submission count"**, with an absolute hard cap of **10 attempts** per 24 hours. When a contestant successfully builds an image via the source-code upload method or successfully pushes an image via the image-push method, a scoring task shall be triggered immediately, consuming one submission attempt at that moment.
- **Quota 2: "Scoring opportunities"**, with an absolute hard cap of **3 results** per 24 hours. A scoring opportunity shall be consumed only when a submission successfully enters the evaluation pipeline and produces a final score.
- Quota deduction rules are as follows:
  - Cases where service startup is not triggered — such as image-build failure under the source-code upload method, or generating temporary credentials but not pushing the image under the image-push method — shall not deduct submission count.
  - Cases entering the scoring stage but failing to produce a score — such as service startup errors, no response on the `/health` endpoint within an absolute hard cap of **300 seconds**, or `/step` endpoint call failures — shall deduct only the submission count and shall not deduct scoring opportunities.
  - Successful scoring (including cases where wall-clock timeout results in a score of zero) shall simultaneously deduct both the submission count and the scoring opportunity.
  - Once either quota is exhausted, no further submissions shall be permitted, and the contestant must wait for the next 24-hour cycle to refresh.
- **Other restrictions:** For each leaderboard, each Team shall have a maximum of one task running concurrently. The backend evaluation system shall automatically evaluate based on the Team's current level. If the Team's score at the current level exceeds the level's advancement threshold, the Team shall automatically advance to the next level, and both the submission count and scoring opportunities shall be reset.

ALIYUN COMMITTED

### Submission Records

- Under the source-code upload method, a submission record shall be generated only after the image-build step succeeds; otherwise, no record shall be generated.
- Under the image-push method, a submission record shall be generated only after the image push succeeds; otherwise, no record shall be generated.

After successful submission, you may view your submitted images, robot configuration, and scores in the submission records below the "Submission Entry" tab. You may also view the submission records of all members of your Team under the "History" tab.

## Computing Resources

The GPU server used to run the contestant's competition image is configured as follows: 16 CPU cores, 24 GB memory, 500 GB disk, with a single-card GPU **RTX 5880 (total VRAM 48 GB)**.

**Available VRAM: approximately 34 GB** (simulation occupies approximately 14 GB). VRAM note: the GPU VRAM is shared between the contestant container and the co-located simulation environment container. The simulation environment is estimated to occupy approximately 14 GB, and the contestant's actual usable VRAM is approximately 34 GB. Please size the model appropriately when designing the solution to avoid task failure due to out-of-memory (OOM) conditions.

The runtime limit is based on real-world physical (Wall Clock) time, with an absolute hard cap of **30 minutes (UTC+8 reference clock)** counted from the start of task execution (excluding image-build time and task-scheduling time). After timeout, partial-step scores shall be granted based on task completion, and one of the day's scoring opportunities shall be consumed.

**Network restriction:** The contestant container is strictly and absolutely prohibited from accessing the internet. All dependencies and model files must be packaged in full at the image-build stage.

AIEC2026 COMMITTEE

## Appendix: Technical Recommendations (Non-Scoring Items)

**Task A:**

The reinforcement-learning-based example method is already capable of basic flat-terrain walking. Subsequent training under diverse terrains and domain randomization can improve the robot's performance on other terrains.

**Task E:**

The ACT-based example method is already capable of grasping and placing single objects. Subsequently, diverse data may be collected to improve the policy's generalization and precision.

**Task B (Locomotion–Manipulation):**

This task requires coordinated locomotion and manipulation. Recommended methods:

- End-to-end reinforcement-learning methods, such as Deep Whole-Body Control
- Hybrid methods: an RL-based control policy for base control combined with classical control methods such as IK / MPC for arm control
- Optimal control: whole-body Model Predictive Control to coordinate the legs and the arm

**Task D (Vision-Based Traversal):**

This task requires the robot to possess a degree of spatial understanding and reasoning capability. Recommended approaches:

- Incorporate image and depth information on top of the control policy to achieve end-to-end control, e.g., Extreme Parkour

**Advanced directions:**

- Vision-Language-Action (VLA)
- Vision-Language Models (VLM)

ATEC2026 COMMITTEE

## Appendix: List of Pre-installed Dependencies in the Environment

```
accelerate==1.2.1
accelerating-doc==0.0.4
annotated-types==0.7.0
anyio==4.13.0
asttokens==3.0.1
certifi==2024.12.14
charset-normalizer==3.3.2
click==8.3.3
decorator==5.2.1
deepspeed==0.16.9+ali
einops==0.8.0
executing==2.2.1
fastapi==0.136.0
filelock==3.15.4
fsspec==2024.6.0
h11==0.16.0
hjson==3.0.2
huggingface_hub==0.30.2
idna==3.7
ipython==9.12.0
```

```
ipython_pygments_lexers==1.1.1
jedi==0.19.2
Jinja2==3.1.4
MarkupSafe==2.1.5
matplotlib-inline==0.2.1
mpmath==1.3.0
msgpack==1.1.0
networkx==3.3
ninja==1.11.1.1
numpy==2.1.3
packaging==24.0
parso==0.8.6
pexpect==4.9.0
pillow==10.3.0
pip==24.2
prompt_toolkit==3.0.52
psutil==5.9.2
ptyprocess==0.7.0
pure_eval==0.2.3
py-cpuinfo==9.0.0
pydantic==2.10.4
pydantic_core==2.27.2
python-multipart==0.0.26
Pygments==2.20.0
PySocks==1.7.1
PyYAML==6.0.1
regex==2024.5.15
requests==2.32.4
safetensors==0.4.3
setuptools==80.9.0
stack-data==0.6.3
starlette==1.0.0
sympy==1.13.3
tokenizers==0.21.0
torch==2.7.1.8+cu128
torchaudio==2.7.1.8+cu128
torchvision==0.22.1.8+cu128
tqdm==4.66.4
traitlets==5.14.3
transformers==4.53.1
triton==3.2.0
typing_extensions==4.12.2
typing-inspection==0.4.2
urllib3==2.5.0
uvicorn==0.45.0
wcwidth==0.6.0
wheel==0.45.0
```

AIEC2026 COMMITTEE

**Common Failure Modes**

The following issues are common causes of task failure or abnormal scores during submission and evaluation. Contestants are advised to inspect them carefully before submission:

### 1. Service Startup Failure (`/health` Timeout)

- **Symptom:** Submission does not enter evaluation, or shows service-startup failure.
- **Causes:**
  - Initialization time too long (exceeding the absolute hard cap of **300 seconds**)
  - Model loading or dependency installation blocking startup
- **Recommendations:**
  - Streamline the initialization process (avoid complex computations during startup)
  - Pre-load models and optimize loading time
  - Ensure the `/health` endpoint responds quickly

### 2. Dependency Conflicts or Environment Incompatibility

- **Symptom:** The program crashes or produces no output during execution.
- **Causes:**
  - Version conflicts with platform-preinstalled libraries (e.g., PyTorch, CUDA)
  - Use of unbundled external dependencies (the platform has no network access)
- **Recommendations:**
  - Avoid overriding core dependencies in the base environment
  - All dependencies must be installed at the image-build stage
  - Test locally against the official environment configuration as closely as possible

### 3. Out of Memory (OOM)

- **Symptom:** The program is interrupted or unresponsive during execution.
- **Causes:**
  - Model too large or inference overhead too high
  - Batch size or intermediate caches not properly controlled
- **Recommendations:**
  - Control model size and inference overhead
  - Avoid unnecessary intermediate-variable storage
  - Reserve sufficient VRAM (the simulation environment already occupies part of the resources)

### 4. Invalid Action Output

ATEC2026 COMMITTEE

- **Symptom:** Abnormal robot behavior, very low scores, or task failure.
- **Causes:**
  - Output dimensionality mismatch
  - Unreasonable control-value ranges (excessively large or unstable)
- **Recommendations:**
  - Strictly check action dimensionality and joint order
  - Clip or normalize outputs
  - Avoid high-frequency oscillating control signals

**5. Slow Inference (Step Timeout)**

- **Symptom:** Tasks terminate early or fail to complete.
- **Causes:**
  - Single-step inference takes too long
  - Use of complex models (such as large-scale VLMs / Transformers)
- **Recommendations:**
  - Optimize inference speed (model compression, computation reduction)
  - Avoid heavy-compute logic inside `/step`
  - Keep total runtime within the prescribed limit

**6. Incomplete Task Logic (Early Termination)**

- **Symptom:** Tasks end prematurely or scores are abnormally low.
- **Causes:**
  - Erroneously triggering `giveup=True`
  - Failing to handle key task phases (e.g., placement or obstacle crossing)
- **Recommendations:**
  - Ensure the full task pipeline is executed
  - Use early-termination logic with caution

**7. Inconsistency Between Simulation and Local Results**

- **Symptom:** Local execution is normal, but online execution fails or scores drop significantly.
- **Causes:**
  - Inconsistency between local and platform environments
  - Failure to account for randomness or edge cases
- **Recommendations:**
  - Use the official environment for local validation
  - Strengthen policy robustness (avoid overfitting specific situations)
  - Test multiple initial conditions and random perturbations

**Recommendation:** Before submission, contestants are advised to complete at least one full local-validation run and inspect the log output and resource usage to ensure that the solution operates stably in the evaluation environment.

ATEC2026 COMMITTEE

## Platform Support and Assurance

- **Platform-fault compensation:** If a contestant's task is abnormally terminated or the evaluation result is abnormal due to force majeure or a fault of the ATEC evaluation platform itself (including but not limited to server downtime, abnormal interruption of the simulation environment, or scoring-system errors), the contestant may submit feedback through the support channel below. After verification and confirmation by the platform, the corresponding submission count and/or scoring opportunities shall be returned. Task failures caused by the contestant's own code errors, dependency conflicts, resource overruns, or similar reasons are strictly and absolutely prohibited from being included in the compensation scope.
- **Technical support channel:** For issues regarding environment configuration, image submission, or evaluation results, please submit feedback uniformly via the Help Center entry on the homepage of the ATEC2026 official competition website. For rule disputes or computing-credit issues, you may also email ATECservice@atecup.com. When submitting feedback, please attach the Team name and Team Leader account, the corresponding submission record number, and a specific description of the issue together with screenshots (if any), so as to enable rapid problem localization.

ATEC2026 Committee

ATEC2026 COMMITTEE